Probabilistic SMS Spam Detection

Project report for Probability and Statistics

BAMAT207 | Probability and Random Variables

Submitted by
YUVRAJ CHANDAK
Registration number 25BCE2561

Submitted to
DR. VEERAMANI S

VELLORE INSTITUTE OF TECHNOLOGY
VIT Vellore

Submission date 27 October 2026
Analysis completed 26 October 2026

Abstract

This project implements a Python system that classifies an SMS as spam or genuine text and displays estimated probabilities for both classes. It connects empirical probability, Bayes' theorem and a Bernoulli random variable with a Multinomial Naive Bayes classifier. The official UCI source contains 5,574 records. After checking missing values and removing 414 duplicate messages following whitespace trimming, 5,160 messages remain. Five suspicious words are analysed using message-level conditional probabilities, and each result is verified through Bayes' theorem.

A stratified 80/20 split produces 4,128 training messages and 1,032 test messages. A CountVectorizer and MultinomialNB pipeline achieves 98.26% test accuracy, 95.08% spam precision and 90.625% spam recall. The model misses 12 spam messages and incorrectly flags 6 genuine messages. A desktop interface, command-line predictor and executed notebook support a live classroom demonstration. Results are reproducible using the saved dataset, split manifest and model. The probabilities are model estimates and should not be interpreted as calibrated certainty.

Project repository
https://github.com/YuvrajC7/sms-spam-detection

1 Project scope and report guide

Problem and objective

Unwanted SMS messages often advertise prizes, offers or paid services. A useful classifier must recognize those patterns without treating every ordinary message containing a word such as “free” as spam. The objective is to build a working educational system, explain its probability calculations, measure its errors on unseen messages and demonstrate predictions interactively.

The project follows the prescribed CountVectorizer and Multinomial Naive Bayes approach. The original messages are retained for traceability; the feature extractor performs its own lowercase tokenization. The system does not contact message senders, open links or automatically delete any SMS. Classification is performed locally.

Report navigation

Section	Subject	Page
1	Scope and report guide	2
2	Dataset provenance and cleaning	3
3	Class probabilities	4
4	Word probabilities and Bayes theorem	5
5	Interpreting the five word events	6
6	Bernoulli random variable	7
7	Data preparation and model	8
8	Evaluation results	9
9	Error analysis	10
10	Live prediction and explanation	11
11	Implementation and reproducibility	12
12	Validation and conclusions	13
13	References and deliverable mapping	14

Success criteria

A successful submission must load and clean the UCI corpus, show class counts and probabilities, calculate all five requested conditional probabilities, derive expectation and variance, train on a stratified 80% subset, evaluate on the remaining 20% and accept a new SMS. It must also provide a notebook or Python source, a trained model, charts, a performance report and a GitHub repository link.

This report is a separate submission artifact from the executable project. Tables and figures are generated from the saved analysis results so that the written evidence can be checked against the code. The repository address identifies the intended submission location; publication is a separate operational step from local execution.

2 Dataset provenance and cleaning

The source is the SMS Spam Collection by Almeida and Hidalgo, distributed through the UCI Machine Learning Repository [1]. Each source record contains a label followed by a tab and the message text. The downloadable source is saved locally, along with its original readme, so subsequent classroom runs do not require a network connection.

Parsing and integrity

Each nonblank source line is split at its first tab. This preserves quotation marks inside messages rather than interpreting them as CSV syntax. Labels are checked against the permitted values ham and spam. Missing fields and empty text are counted. The program rejects conflicting labels for the same cleaned message instead of silently choosing one. Every retained record stores its original source line and a message identifier.

Cleaning check	**Observed result**
Raw labelled records	5574
Raw ham and spam	4,827 ham and 747 spam
Missing label or message rows	0
Empty messages after trimming	0
Conflicting duplicate labels	0
Removed duplicate message rows	414
Retained unique messages	5160
Final ham and spam	4,518 ham and 642 spam

Duplicate policy

Leading and trailing whitespace is removed from message text before duplicate comparison. The first occurrence of each identical resulting message is retained. Duplicate removal is based on the message, not the entire row including its source line. Case differences and internal whitespace differences are preserved. Thus the policy is exact text deduplication after trimming, not semantic or template deduplication.

Duplicate removal occurs before the train-test split. Otherwise an identical SMS could appear in both partitions and make the test score look stronger than it is. Near-duplicate promotional templates may still span the partitions; this remains a limitation of the evaluation.

Identity and attribution

The source archive is obtained from https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip. UCI lists the dataset under CC BY 4.0 [1]. The distributed data retains attribution in data/ATTRIBUTION.md. The source file checksum is recorded in results/metrics.json, enabling an exact byte-level source check.

SHA256 7d039a24a6083ed9ef0f806ebad56bbb976e3aeb8de05669173bfdc4996c239d

The assignment and UCI describe 5,574 instances, and the line-based parser observes the same total. Counts from other copies or CSV import settings may differ. The results in this report refer only to the exact saved source and cleaning policy above.

3 Class probabilities

Let S denote the event that a uniformly selected message from the cleaned corpus is spam, and H denote ham. These events are mutually exclusive and exhaustive because every retained message has one of the two labels. The probability estimates therefore sum to one.

P(S) = 642 / 5160 = 0.12441860

P(H) = 4518 / 5160 = 0.87558140

Class	Count	Probability	Percentage
Ham	4518	0.87558140	87.56%
Spam	642	0.12441860	12.44%
Total	5160	1.00000000	100.00%

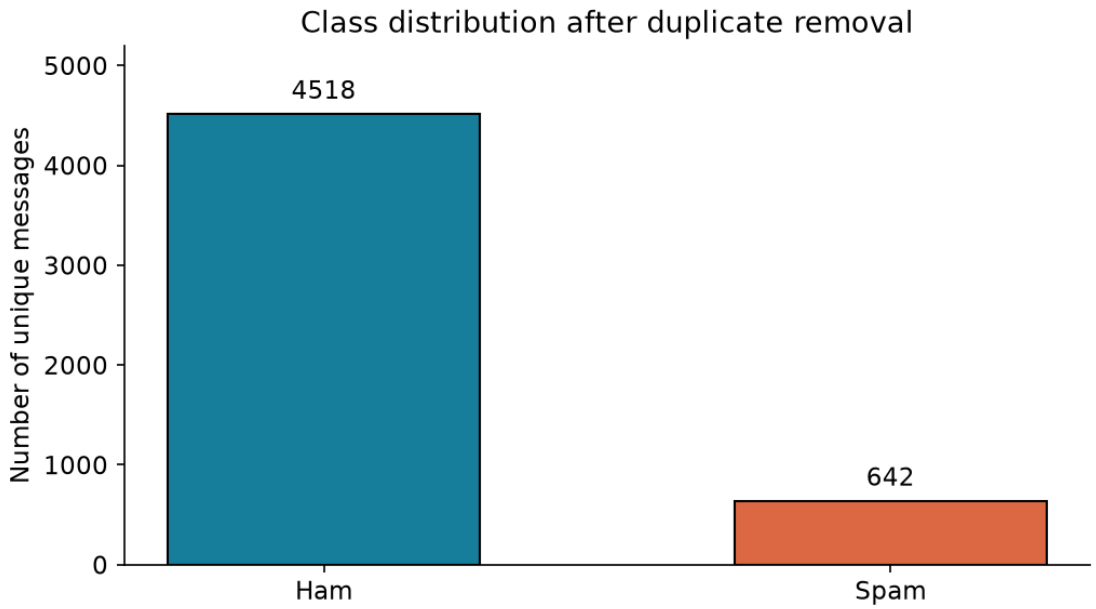


Figure 1. Class counts after trimming and removing exact duplicate messages.

Why class imbalance matters

Ham accounts for about seven out of every eight cleaned messages. Accuracy alone can therefore be misleading: a classifier that predicts ham for everything already gets most messages correct while detecting no spam. The final evaluation includes spam precision, spam recall and F1 in addition to overall accuracy, and explicitly compares against an always-ham baseline.

These empirical priors describe the cleaned corpus. They do not estimate the current spam prevalence in every phone user's inbox. The learned model uses the training subset's class frequencies, which are close to these proportions but not identical because the stratified split uses integer counts.

4 Word probabilities and Bayes theorem

For a suspicious word w, define W as the event that a message contains w as a whole word, ignoring letter case. The regular expression uses word boundaries on both sides. “FREE” matches “free”; “freedom” does not. Repeated occurrences in one message contribute once to this analysis. This event differs from the token-frequency features used by the classifier.

P(S | W) = n(S and W) / n(W)

P(S | W) = P(W | S) P(S) / P(W)

Word	All with word	Spam	Ham	Direct P(S | W)
free	199	144	55	0.723618
prize	70	70	0	1.000000
win	53	47	6	0.886792
urgent	61	56	5	0.918033
offer	30	24	6	0.800000

Worked example for free

Among 199 messages containing “free”, 144 are spam and 55 are ham. The direct estimate is therefore 144/199. For Bayes’ theorem, the denominator P(Free) uses the entire cleaned corpus, while P(Free | Spam) uses only the 642 spam messages.

P(Free | S) = 144 / 642 = 0.22429907

P(Free) = 199 / 5160 = 0.03856589

P(S | Free) = (144/642)(642/5160) / (199/5160)

P(S | Free) = 144/199 = 0.72361809

The intermediate counts cancel algebraically, so Bayes’ theorem and the direct ratio must agree. The implementation checks equality numerically for all five words. Calculation uses full precision; rounding is applied only for display.

Avoiding a common reversal

P(Spam | Free) is about 72.36%, whereas P(Free | Spam) is about 22.43%. The first asks how likely spam is among messages containing “free”; the second asks how often spam messages contain “free”. They use different sample spaces and must not be interchanged.

The five words were prescribed by the assignment. Their full-corpus descriptive analysis is not fed into feature selection or model tuning. The trained classifier receives only training messages and labels. A word absent from the corpus would have an undefined conditional probability, not a probability of zero.

5 Interpreting the five word events

Word	P(W | S)	P(W)	Bayes P(S | W)
free	0.224299	0.038566	0.723618
prize	0.109034	0.013566	1.000000
win	0.073209	0.010271	0.886792
urgent	0.087227	0.011822	0.918033
offer	0.037383	0.005814	0.800000

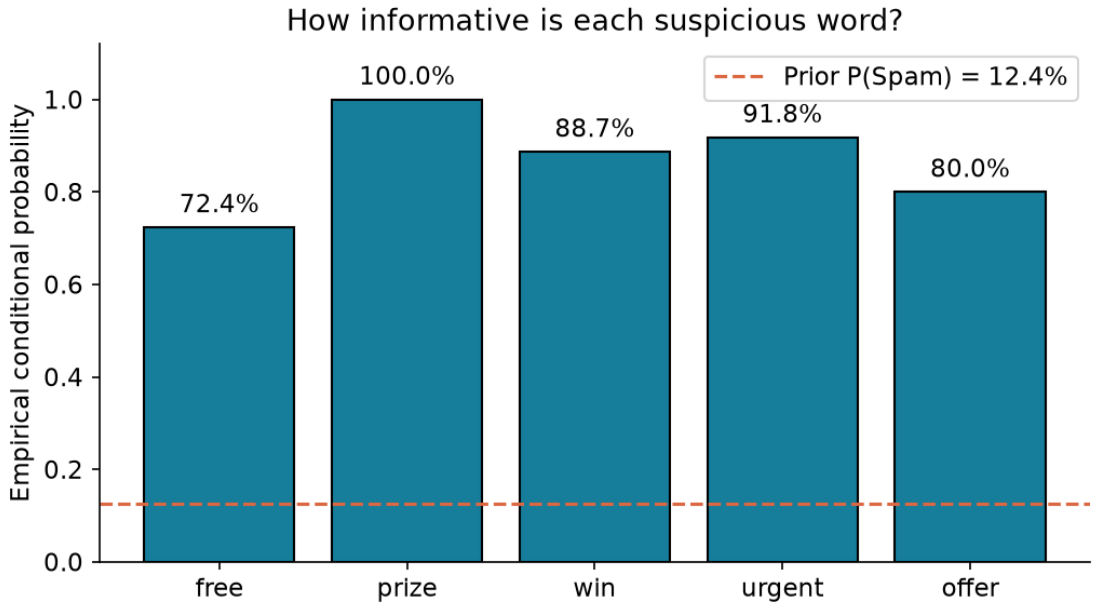


Figure 2. Conditional spam proportions for the five required words. The dashed line is the corpus spam prior.

“Prize” occurs in 70 retained messages, all labelled spam, giving an observed proportion of 1.000000. This finite-sample result does not prove that every future message containing “prize” is spam. It also does not override other evidence in a full-message model prediction.

“Urgent” appears in 61 messages, of which 56 are spam: 56/61 = 0.918033. “Win” appears in 53 messages, with 47 spam: 47/53 = 0.886792. “Offer” appears in 30 messages, with 24 spam: 24/30 = 0.800000. The different denominators matter because a proportion supported by fewer observations is less stable.

“Free” provides a useful classroom counterexample. Its observed spam proportion is substantially above the 12.44% prior, yet 55 messages containing it are genuine. A phrase such as “Are you free for lunch tomorrow?” shows why a single-word rule is insufficient. The classifier combines evidence from all recognized tokens rather than using this five-word table as a rule list.

The event counts are descriptive associations, not causal effects. The words do not cause the labels, and the dataset selection process influences the observed relationships. These tables quantify this corpus rather than establishing general laws of language.

6 Bernoulli random variable analysis

Define Y as an indicator of spam for a randomly selected cleaned message. Y takes only the values 0 and 1. A random variable with this two-outcome indicator structure follows a Bernoulli distribution under the empirical sampling model.

Y = 1 for spam and Y = 0 for ham

Y ~ Bernoulli(p), where p = 642/5160

Value y	Interpretation	P(Y = y)
0	Ham	0.87558140
1	Spam	0.12441860

Expected value

E(Y) = sum of y P(Y = y)

E(Y) = 0(1-p) + 1(p) = p = 0.12441860

The expected value is the long-run fraction of spam under repeated uniform sampling from the empirical corpus. It does not mean that an individual message is “0.1244 spam”; each observed label is still either zero or one. The average of the 5,160 encoded labels equals this expectation exactly.

Variance

Y squared = Y, so E(Y squared) = p

Var(Y) = E(Y squared) - [E(Y)] squared

Var(Y) = p - p squared = p(1-p) = 0.10893862

Variance quantifies dispersion of the binary indicator around its mean. The product p(1-p) is largest at p=0.5 and becomes zero when all labels belong to one class. For this ham-dominated sample, the observed variance is below the maximum Bernoulli variance of 0.25.

Verification and interpretation

The notebook compares p(1-p) with the numerical variance of the encoded labels using ddof=0. This treats the cleaned corpus as the empirical distribution being described. Using the unbiased sample variance with ddof=1 would introduce a factor of n/(n-1) and would answer a different estimation question.

As an illustration, 100 independent draws with replacement from this empirical distribution would have an expected spam count of 100p = 12.4419 and count variance 100p(1-p) = 10.8939. This extension requires independence and sampling with replacement; it is not a claim about a real sequence of 100 incoming messages.

7 Data preparation and model building

Stratified training and testing

The code uses train_test_split with test_size=0.2, random_state=42 and stratify=target. Stratification approximately preserves the spam fraction in both partitions. The fixed random seed makes the split reproducible for the same ordered input and software environment. The saved split manifest records the membership explicitly.

Partition	Ham	Spam	Total
Training	3614	514	4128
Test	904	128	1032
Total	4518	642	5160

Text representation

CountVectorizer is fitted only on training text and learns 7,673 vocabulary terms. Its default analyzer lowercases text and selects tokens containing at least two word characters. Punctuation acts as a separator. The output is a sparse nonnegative word-count matrix: each row is a message, each column is a vocabulary term and each value counts that term in the message. No stemming, stopword filtering or TF-IDF transformation is applied [2].

The test messages are transformed with the already learned vocabulary. Unknown terms are ignored. A Pipeline stores the vectorizer and classifier together so live input undergoes the same transformation as training and test data. No vocabulary fitting is performed on the complete corpus before splitting.

Multinomial Naive Bayes

The classifier models word-count evidence for each class. Let N(c,j) be the total training count of vocabulary term j in class c, V be the vocabulary size and alpha=1. Laplace smoothing assigns a positive estimated probability even when a term has not appeared in one class [3].

theta(c,j) = [N(c,j) + 1] / [sum over k N(c,k) + V]

score(c) = log P(c) + sum over j x(j) log theta(c,j)

The class prior is estimated from the training labels: P_train(Spam)=514/4128=0.12451550. The two class scores are normalized to produce predict_proba outputs. Computing in log space avoids multiplying many small probabilities directly. The predicted label is the class with the larger posterior score; a tie follows the library's class order.

"Naive" refers to the conditional independence assumption for token evidence given the class. This simplifies estimation but ignores word order and relationships such as negation. It is a suitable, interpretable baseline for the assigned discrete text features. The default model is used without test-driven tuning.

8 Evaluation on held out messages

All performance values below use the 1,032 test messages, which were excluded from fitting the vocabulary and classifier. Spam is the positive class. In Figure 3, rows are actual labels and columns are predicted labels, with ham first and spam second.

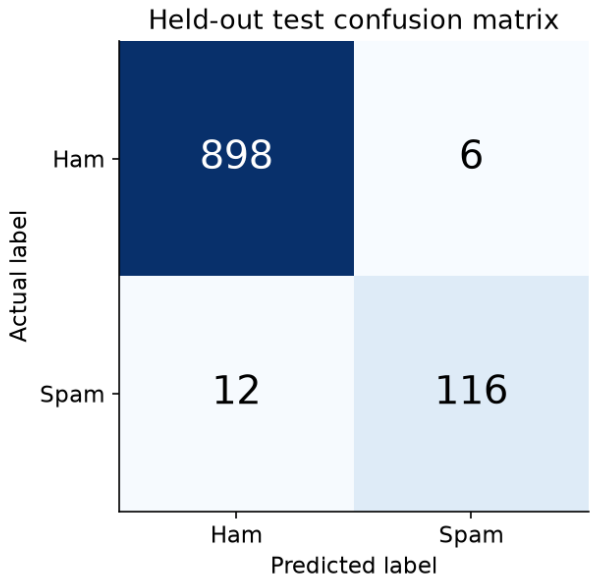


Figure 3. Confusion matrix on the fixed held-out test set.

Class or average	Precision	Recall	F1 score	Support
Ham	0.9868	0.9934	0.9901	904
Spam	0.9508	0.9062	0.9280	128
Macro average	0.9688	0.9498	0.9590	1032
Weighted average	0.9823	0.9826	0.9824	1032

Accuracy = (TN + TP) / N = (898 + 116)/1032 = 98.26%

Spam precision = TP/(TP+FP) = 116/122 = 95.08%

Spam recall = TP/(TP+FN) = 116/128 = 90.625%

Spam F1 = 2TP/(2TP+FP+FN) = 232/250 = 92.80%

The always-ham baseline obtains 904/1032 = 87.60% accuracy and zero spam recall. The model improves accuracy by 10.66 percentage points and detects 116 of the 128 test spam messages. Macro averages weight the two classes equally; weighted averages weight them by their test supports. The latter are dominated by ham because it is more common.

9 Understanding classification errors

False negatives and false positives

There are 12 false negatives: actual spam messages predicted as ham. They represent 12/128 = 9.375% of test spam. In a deployed filter, such messages would still reach the inbox. There are 6 false positives: genuine messages predicted as spam, corresponding to 6/904 = 0.664% of test ham. These errors could hide useful communications, so they are important even though they are numerically fewer.

Examples from the held out errors

Source line 4863 and false positive

Message excerpt: Nokia phone is lovly..

Actual label: ham. P(Spam | Message) = 0.937889. This genuine text mentions a phone brand. Brand and product terms can resemble promotional language. The model gives spam high probability despite the short ordinary statement.

Source line 3891 and false positive

Message excerpt: Unlimited texts. Limited minutes.

Actual label: ham. P(Spam | Message) = 0.859556. The compact wording resembles a telecom offer. With little surrounding context, a bag-of-words model has limited evidence to distinguish conversation from advertising.

Source line 3361 and false negative

Message excerpt: Sorry I missed your call let's talk when you have the time. I'm on [number omitted]

Actual label: spam. P(Spam | Message) = 0.000002. The labelled spam message imitates a personal conversation and a missed-call reply. Familiar conversational words can outweigh the suspicious contact pattern.

Source line 4257 and false negative

Message excerpt: Block Breaker now comes in deluxe format with new features and great graphics from T-Mobile. Buy for just £5 by replying GET BBDELUXE and take the challenge

Actual label: spam. P(Spam | Message) = 0.006627. The labelled spam promotes a paid mobile game. Its particular vocabulary is not enough for this model to assign spam the larger probability. The training corpus does not guarantee coverage of every promotional style.

What these examples establish

These explanations describe plausible linguistic patterns, not experimentally proven causes. The exact model contribution of each recognized token can be inspected with the predictor. A confident error is possible because Naive Bayes probabilities are not necessarily calibrated. All 18 errors, their original source lines, labels and estimated spam probabilities are saved in results/errors.csv for audit.

The error set is inspected after fitting to understand weaknesses. It is not used to change the model or choose a threshold in this submission. Future improvements should be selected on training-only validation data, reserving a fresh test set for final evaluation.

10 Live prediction and explanation

Demonstration result

Input SMS: “Congratulations! You have won a free prize. Claim now.”

Prediction = Spam

P(Spam | Message) = 0.9999998019

P(Ham | Message) = 0.0000001981

The two probabilities sum to one up to floating-point rounding. The assignment’s sample values of 0.96 and 0.04 are illustrative; the values above are computed from the saved fitted pipeline. The result is highly confident under this model, but it does not establish certainty that an arbitrary future message is spam.

How the explanation is calculated

For each known term, the interface displays its exact additive contribution to spam-versus-ham log odds: count × [log theta(Spam,word) - log theta(Ham,word)]. Positive values favor spam; negative values favor ham. The prior log odds must also be included. These contributions describe the model calculation, not causal influence.

Term	Count	Log odds contribution
claim	1	+5.3448
prize	1	+5.1705
congratulations	1	+3.3299
free	1	+2.3459
won	1	+2.0490
you	1	-0.9552
now	1	+0.4472
have	1	-0.3475

The prior contributes -1.9503. Adding all token contributions gives the final log odds, and the logistic transformation gives P(Spam | Message). The automated checks verify this reconstruction against predict_proba. The interface shows the strongest terms first; the command-line JSON exposes every term.

Classroom interaction

Run START_DEMO.cmd on Windows, then use the prize example, an ordinary library message, and “Are you free for lunch tomorrow?”. Enter an original message and select Analyse message, or press Ctrl+Enter. A second tab presents dataset counts, Bernoulli quantities, word probabilities and evaluation metrics.

Empty input is rejected with a clear message. If no tokens are recognized, the interface warns that the probabilities reduce to the training priors. This is absence of usable vocabulary evidence, not a reliable judgement of safety. Input is limited to 10,000 characters to keep the classroom interface responsive.

11 Implementation and reproducibility

Project structure

File or folder	Purpose
spam_detector.py	Loading, cleaning, probabilities, training, evaluation and prediction
demo.py	Offline desktop interface for classroom presentation
SMS_Spam_Detection.ipynb	Executed notebook explaining all eight activities
data/	Official source, attribution and cleaned CSV
models/spam_pipeline.joblib	Fitted vectorizer and classifier in one pipeline
results/	Metrics, charts, word calculations, split manifest and all test predictions
tests/test_project.py	Automated correctness and consistency checks
DEMO_GUIDE.md	Presentation sequence and viva preparation

Reproduce the results

Use Python 3.12 and the pinned requirements for the closest match to the recorded environment. From the project directory, install dependencies, run the training command and execute the tests. The saved source allows training to work offline once dependencies are available. If the source file is absent, the program downloads the official archive with a timeout.

```
python -m pip install -r requirements.txt
```

```
python spam_detector.py train
```

```
python -m unittest discover -s tests -v
```

```
python demo.py
```

```
python spam_detector.py interactive
```

The notebook can be opened in Jupyter or uploaded with the project files to Colab. Execute cells in order. To reproduce the published measurements exactly, preserve the source checksum, cleaning policy, ordered rows, split seed and dependency versions. The saved model is the same training-only model used in evaluation; it is not subsequently refitted on the full dataset.

Recorded environment

python 3.12.14; sklearn 1.9.1; pandas 3.0.1; numpy 2.3.5; joblib 1.6.0.

The model artifact uses joblib serialization. Only load the locally generated artifact or another trusted copy; model files are executable Python serialization, not a safe interchange format for unknown downloads. The project does not log new live-demo messages to disk.

12 Validation limitations and conclusions

Verification performed

Six automated tests pass. They check the cleaning reconciliation, uniqueness and missing values; whole-word counting with a hand-verifiable fixture and Bayes equality; disjoint train/test membership and training-only vocabulary; probability normalization and exact log-odds reconstruction; empty, invalid and overlong input; and agreement between the saved model, saved test predictions and confusion matrix. The notebook is executed from start to finish, and the desktop callbacks are exercised with prize, ordinary, ambiguous and unknown-vocabulary inputs.

Limits of the evidence

The measured accuracy belongs to one fixed random split of a historical English SMS corpus. It does not establish present-day performance on multilingual messages, new scam techniques or a particular institution's inbox. Exact duplicates are removed, but paraphrases and repeated campaign templates may remain. A group-based or time-based evaluation would better test some forms of generalization if appropriate metadata were available.

The model ignores word order and treats token evidence as conditionally independent. Correlated words can lead to overconfident probabilities. No probability calibration is fitted, and no calibration guarantee is made. Tokens outside the training vocabulary are ignored; a message with no recognized tokens receives training priors. Extremely short messages can be difficult because they supply little context.

The full-corpus word analysis serves the probability assignment, while performance evaluation uses a training-only model. A deployment-oriented analysis would estimate word statistics within training folds and evaluate their stability. Current false-negative and false-positive rates are descriptive test rates; no repeated-split uncertainty interval is claimed.

Future work

Reasonable next experiments include character n-grams for obfuscated spelling, bigrams for short phrases, training-only cross-validation for smoothing strength, and calibrated probabilities using held-out validation folds. Threshold selection should reflect the cost of missed spam versus blocked genuine messages. These are proposed improvements, not experiments performed in this submission.

Conclusion and learning outcome

The completed system links elementary probability with a functioning text classifier. Empirical class probabilities provide priors, word events illustrate conditional probability and Bayes' theorem, and the spam indicator demonstrates Bernoulli expectation and variance. Word-count features and Multinomial Naive Bayes extend these ideas to complete messages. The held-out accuracy of 98.26% is supported by a confusion matrix and class-specific metrics rather than accuracy alone.

The executable demo, notebook, saved model and analysis outputs make the calculations inspectable during presentation. The project also provides a reproducible repository structure and attribution suitable for a public academic portfolio. Its strongest conclusion is that a simple probabilistic model performs well on this controlled dataset while still making identifiable, explainable errors.

13 References and submission mapping

References

[1] Almeida, T. and Hidalgo, J. (2011). SMS Spam Collection [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5CC84. https://archive.ics.uci.edu/dataset/228/sms+spam+collection. Dataset license: CC BY 4.0.

[2] scikit-learn developers. CountVectorizer API reference. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html.

[3] scikit-learn developers. Naive Bayes user guide, including Multinomial Naive Bayes. https://scikit-learn.org/stable/modules/naive_bayes.html.

[4] scikit-learn developers. train_test_split API reference. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html.

[5] Course assignment. BAMAT207 Probability and Statistics, Project Activity, Probabilistic SMS Spam Detection. Supplied Project_activity.docx. Submission deadline 27 October 2026.

Online documentation and dataset accessed on 26 October 2026. All reported numerical results are generated by this project from the saved UCI source; software documentation supports the method descriptions.

Assignment deliverables

Required deliverable	**Location in the submission**
Python or Colab notebook	project/SMS_Spam_Detection.ipynb and spam_detector.py
Cleaned dataset summary	Report sections 2 and 3; data/cleaned_sms.csv; results/metrics.json
Class distribution chart	project/results/class_distribution.png
Five suspicious word probabilities	Report sections 4 and 5; results/word_probabilities.csv
Trained Naive Bayes model	project/models/spam_pipeline.joblib
Confusion matrix and performance report	project/results/confusion_matrix.png and classification_report.txt
Interactive SMS prediction	project/demo.py and command-line interactive mode
Report in editable and final formats	report/SMS_Spam_Detection_Report.docx and .pdf
GitHub submission address	https://github.com/YuvrajC7/sms-spam-detection

Presentation readiness

Before class, launch the desktop demo, read the worked Bayes example and verify the three central numbers: 5,160 cleaned messages, 642 spam messages and 1,032 held-out test messages. Be ready to distinguish empirical word probabilities from model posteriors, explain both error types and show why the always-ham baseline is insufficient. The separate demo guide provides a timed presentation sequence and short viva answers.